

Optimizing API Gateway Performance with Serverless

Architectures: A Comprehensive Analysis

<https://ijsrcseit.com/index.php/home/article/view/CSEIT251112276/CSEIT251112276>

- 2025

1. Qual é o problema de pesquisa que o artigo busca resolver?

O artigo parte de um ponto bem estratégico dentro de sistemas modernos: o API Gateway, funciona como a porta de entrada de tudo.

O problema é que, à medida que o sistema cresce e o volume de requisições aumenta, esse gateway pode virar um gargalo sério. Ele começa a introduzir latência, aumenta o tempo de resposta e, em casos mais críticos, pode até comprometer a estabilidade do sistema inteiro.

Além disso, existe um problema paralelo: manter infraestrutura tradicional rodando o tempo todo é caro, mesmo quando o sistema não está sendo muito utilizado. Então o artigo tenta resolver dois pontos ao mesmo tempo:

- melhorar o desempenho do API Gateway sob carga variável
- reduzir custos operacionais sem perder eficiência

A proposta surge justamente nesse conflito entre desempenho e custo.

2. Qual é o contexto ou domínio de aplicação do trabalho?

O trabalho está totalmente inserido no universo de arquiteturas cloud-native, onde aplicações são construídas para rodar na nuvem de forma distribuída, escalável e resiliente.

Dentro disso, ele foca em dois pilares bem atuais:

- API Gateways, que organizam e controlam o fluxo de requisições
- Serverless computing, onde você não gerencia servidores diretamente

Esse tipo de arquitetura é muito usado em:

- plataformas de streaming

- sistemas financeiros
- aplicações SaaS
- qualquer sistema com tráfego variável

Ou seja, conversa diretamente com o tema, porque APIs são o coração desses sistemas.

3. Qual é a principal pergunta de pesquisa ou hipótese levantada pelos autores?

Questiona se usar uma arquitetura serverless para o API Gateway realmente melhora o desempenho e reduz custos em comparação aos modelos tradicionais.

A hipótese dos autores é que o serverless consegue se adaptar melhor a cenários dinâmicos, porque ele não depende de infraestrutura fixa.

Em vez de manter servidores sempre ativos, o sistema nasce quando precisa responder e some quando não é necessário.

4. Quais são os métodos ou abordagens utilizadas (algoritmos, modelos, técnicas)?

A abordagem principal é:

- implementar um API Gateway usando serverless
- comparar com um modelo tradicional baseado em servidores provisionados

Eles analisam como cada arquitetura se comporta em diferentes situações, observando principalmente:

- escalabilidade automática
- tempo de resposta
- consumo de recursos

É uma análise bem prática, mais voltada para engenharia de sistemas do que para modelagem matemática.

5. Quais dados ou bases de dados foram utilizados no estudo?

O artigo não depende de uma base de dados clássica (ex. dataset público), mas sim de cenários de teste controlados que simulam uso real.

Eles trabalham com:

- diferentes volumes de requisições
- padrões de tráfego variáveis (picos e quedas)
- cenários de uso contínuo x intermitente

Esses dados são gerados ou coletados durante os experimentos, tentando reproduzir o comportamento de aplicações reais.

6. Como foi feito o desenho do experimento ou da avaliação?

O experimento um comparativo direto entre duas arquiteturas:

1. API Gateway tradicional (com servidores provisionados)
2. API Gateway baseado em serverless

Eles submetem essas duas abordagens a diferentes cenários de carga:

- baixa demanda
- picos repentinos
- carga constante

A ideia é ver não só quem é mais rápido, mas quem se adapta melhor ao longo do tempo.

7. Quais métricas de avaliação foram utilizadas para medir os resultados?

As métricas são bem focadas em performance e custo:

- latência: quanto tempo demora para responder
- tempo de resposta geral: experiência do usuário
- escalabilidade: capacidade de lidar com aumento de carga
- custo: quanto se paga pela infraestrutura

8. Quais foram os principais resultados ou conclusões do artigo?

Os resultados mostram que o serverless tem um comportamento muito interessante: em cenários de carga variável, ele se adapta melhor, reduz custos quando o sistema não está sendo muito usado e consegue escalar rapidamente quando há aumento de demanda

Mas por outro lado, existe o problema do cold start (quando a função demora um pouco pra iniciar) e em cargas muito constantes e altas, o ganho pode diminuir

A conclusão geral é que serverless é uma ótima solução para cenários dinâmicos, mas em qualquer situação.

9. Quais são as limitações apontadas pelos autores?

- cold start pode impactar a latência inicial
- dependência de provedores de nuvem (vendor lock-in)
- menor controle sobre a infraestrutura
- dificuldade de ajuste fino em comparação com servidores tradicionais

Ou seja, você ganha automação e flexibilidade, mas perde um pouco de controle técnico.

10. Que oportunidades de pesquisa futura ou lacunas o artigo indica?

- reduzir ou eliminar o impacto do cold start
- criar arquiteturas híbridas (serverless + tradicional)
- melhorar o roteamento e processamento dentro do gateway
- integrar com técnicas de inteligência artificial para otimização dinâmica

11. O que deste artigo pode contribuir ou inspirar o seu trabalho?

Esse artigo é muito valioso, porque enquanto outros trabalhos falam de sistema como um todo, esse mostra que só otimizar só os micro serviços não é suficiente e a entrada do sistema também pode ser um gargalo crítico